

บทที่ 3

Document Type Definition (DTD)

3.1 การตรวจสอบความถูกต้องของเอกสาร XML

ความถูกต้องของเอกสาร XML จะมีอยู่ 2 รูปแบบคือ Well-Formed XML และ Valid XML :

- Well-Formed XML การที่เอกสาร XML นั้นจะ well form ได้ก็ต่อเมื่อเอกสาร XML นั้นเขียนได้ถูกต้องตาม syntax ของ XML ได้แก่การปฏิบัติตาม nesting rule การตั้งชื่ออิลิเมนต์ การใส่เครื่องหมายคำพูดครอบค่าของแอตทริบิวต์ ฯลฯ ดังที่ได้กล่าวมาแล้ว
- ส่วน valid XML เอกสาร XML นั้นจะ valid ได้ก็ต่อเมื่อเอกสาร XML นั้นมีการตรวจสอบด้วยวิธีการ DTD (Document Type Definition) หรือ XML Schema

เนื่องจากจุดเริ่มต้นของการมีภาษา XML ขึ้นมานั้นเกิดจากเราต้องการให้ข้อมูลของเราสามารถสื่อความหมายในตัวเองได้ ทำให้เกิดประเด็นที่ต้องพิจารณาว่า ถ้าเรานำ XML มาอธิบายความหมายของข้อมูลแล้ว ตัวเอกสาร XML เองก็ควรมีสิ่งที่จะมาให้ความหมายแก่เอกสาร XML ด้วย ซึ่งในปัจจุบันเราจะใช้ DTD เข้ามาอธิบายเอกสาร XML ของเราว่าเอกสารของเรานั้น ประกอบด้วยอิลิเมนต์อะไรบ้าง อิลิเมนต์ในเอกสารมีลำดับการใช้งานอย่างไร และข้อมูลที่อิลิเมนต์ต่างๆเก็บนั้นคืออะไร

สำหรับ DTD นั้นเป็นวิธีที่การสร้างนิยามให้แก่เอกสารซึ่งถือกำเนิดมาจากภาษา SGML ซึ่งอย่างที่กล่าวไปแล้วว่า XML เป็นภาษาที่เป็นส่วนย่อยหรือเป็นsubsetของภาษา SGML ดังนั้นเราจึงสามารถนำ DTD มาอธิบายเอกสาร XML ได้ DTD ถือว่าเป็นส่วนเพิ่มเติม (optional) ของเอกสาร XML นั่นคือเราจะกำหนดให้เอกสาร XML ของเรามี DTD กำกับอยู่ด้วยหรือไม่ก็ได้ จะกำหนดให้อยู่ภายในเอกสารของเราเลย หรือว่าจะแยกเป็นอีกไฟล์ก็ได้ ถ้าเป็นกรณีหลังเราจะได้ไฟล์ที่มีนามสกุล .dtd เพิ่มขึ้นมาอีกไฟล์หนึ่ง

3.2 องค์ประกอบของ DTD

DTD นั้นเป็นสิ่งที่ให้นิยามหรือเป็นการสร้างข้อตกลงให้แก่เอกสาร XML ของเรา เช่นในเอกสารจะต้องประกอบไปด้วยอิลิเมนต์อะไรบ้าง เป็นต้น ทั้งนี้การกำกับเอกสาร XML ด้วย DTD จะทำให้เอกสาร XML ของเราสามารถมีการใช้ร่วมกับคนอื่นหรือแอปพลิเคชันอื่นได้โดยที่ผู้ที่มาขอใช้ (หรือที่ขอมให้เราใช้) ก็จะต้องมีการปฏิบัติตามข้อกำหนดที่ระบุไว้ใน DTD นั้นเอง

DTD มีองค์ประกอบหลายอย่าง ได้แก่ element, attribute, entity, processing instruction และ comment โดยองค์ประกอบบางอย่างก็เป็นสิ่งที่ DTD ต้องมี แต่บางอย่างไม่จำเป็นต้องมีก็ได้ ซึ่งมีรายละเอียดดังต่อไปนี้

3.3 Document Type Declarations

ในการที่เราจะกำหนด syntax และโครงสร้างของอิลิเมนต์ของ DTD นั้น เราจะต้องประกาศ (declare) ข้อกำหนดในเอกสารโดยใช้ document type declaration โดยเราจะใช้ `<!DOCTYPE>` ในการสร้าง document type declaration ในอิลิเมนต์นี้จะมีรูปแบบการใช้งานอยู่หลายแบบดังนี้

- `<!DOCTYPE rootname [DTD]>`
- `<!DOCTYPE rootname SYSTEM URL>`
- `<!DOCTYPE rootname SYSTEM URL [DTD]>`
- `<!DOCTYPE rootname PUBLIC identifier URL>`
- `<!DOCTYPE rootname PUBLIC identifier URL [DTD]>`

โดยที่ rootname คือ root ของอิลิเมนต์, URL คือ URL ของ DTD

3.3.1 #PCDATA

โดยปกติ XML จะมองเท็กซ์ (text) ทุกตัวเป็น parsable character ก็คือต้องมีความถูกต้องโดยพาร์สเซอร์และเรียกเท็กซ์ประเภทนี้ว่า PCDATA (Parsable Character Data) และเมื่ออิลิเมนต์ใดถูกกำหนดให้เป็น PCDATA แล้ว ข้อความที่อยู่ระหว่างอิลิเมนต์นั้นก็จะถูกพาร์สหรือถูกตรวจสอบโครงสร้างของข้อมูล เช่นถ้าเรากำหนดในส่วนของ DTD ให้อิลิเมนต์ที่ชื่อ To เป็น PCDATA จะได้ดังนี้

`<!ELEMENT To (#PCDATA)>`

3.4 Elements Declarations

สำหรับ DTD เราใช้อิลิเมนต์เพื่อกำหนดรายละเอียดต่างๆของเอกสาร XML ของเรา ภายในอิลิเมนต์หนึ่งๆนั้น จะมีการประกาศชื่อและประเภทของข้อมูลที่สามารถเก็บได้ ซึ่งเราจะเรียกตรงส่วนนี้ว่า “Content Specification”

การประกาศ content specification ดังกล่าวสามารถทำได้ใน 4 ลักษณะดังต่อไปนี้

1. อิลิเมนต์ที่ประกอบด้วยรายการของอิลิเมนต์อื่นๆ ตั้งแต่ 1 ตัวขึ้นไป ตัวอย่างเช่น

`<!ELEMENT Email(To,From,CC,BCC,Subject,Body)>`

ซึ่งมีความหมายว่าอิลิเมนต์ที่ชื่อ Email ประกอบไปด้วยรายการของอิลิเมนต์ต่างๆได้แก่ To, From, CC, BCC, Subject และ Body ซึ่งเราจะเรียกแต่ละอิลิเมนต์ในรายการดังกล่าวว่าเป็น “Sub Element” หรือ “Child Element” ของอิลิเมนต์ Email และเมื่อรวมกันเราจะเรียกว่า “Content Model” ซึ่งแต่ละอิลิเมนต์ใน Content Model จะต้องมีการประกาศรายละเอียดของตัวเองใน DTD ด้วย จากตัวอย่างข้างต้น เมื่อเราประกาศว่าอิลิเมนต์ Email ประกอบไปด้วยอิลิเมนต์ To, From, CC, BCC, Subject และ Body แล้ว เราจะต้องมีการประกาศต่อไปว่า อิลิเมนต์ To, From, CC, BCC, Subject และ Body มีการเก็บข้อมูลหรือเก็บอิลิเมนต์เป็นแบบไหน

2. อิลิเมนต์ที่ไม่มีการเก็บข้อมูลใดๆ โดยจะใช้สัณยักรัศ EMPTY ตัวอย่างเช่น

```
<!ELEMENT X EMPTY>
```

ซึ่งมีความหมายว่า X เป็นอิลิเมนต์ว่าง (empty element) คือเป็นอิลิเมนต์ที่ไม่มีการเก็บข้อมูลใดๆเลยนั่นเอง ถ้าเทียบกับแท็กใน HTML ก็เทียบได้กับแท็ก
 หรือ <HR> ซึ่งเป็นแท็กที่มีไว้เพื่อการใช้งานบางอย่าง แต่ไม่มีข้อมูลใดๆบรรจุอยู่ในแท็กนั้น เช่น เราใช้
 เพื่อขึ้นบรรทัดใหม่ เป็นต้น

3. อิลิเมนต์ที่สามารถเก็บข้อมูลใดๆก็ได้ ตัวอย่างเช่น

```
<!ELEMENT Y ANY>
```

ในกรณีของอิลิเมนต์ Y คือเราอาจไม่แน่ใจว่าข้อมูลที่อิลิเมนต์ Y จะเก็บนั้นเป็นข้อมูลแบบใด หรืออาจจะประกาศในลักษณะดังกล่าวถ้า อิลิเมนต์ Y มีการเก็บข้อมูลหลายๆแบบ และเราไม่ต้องการเจาะลงไปว่า อิลิเมนต์ Y จะเก็บข้อมูลแบบใดบ้าง

4. อิลิเมนต์ที่มีการเก็บข้อมูลแบบผสม กล่าวคือ อิลิเมนต์สามารถเก็บข้อมูลได้อย่างใดอย่างหนึ่งโดยใช้เครื่องหมายไปป์ (pipe, |) เป็นตัวขึ้นข้อมูล ตัวอย่างเช่น

```
<!ELEMENT Z (#PCDATA|X|Y)>
```

ซึ่งหมายความว่า Z เป็นอิลิเมนต์ที่สามารถเก็บข้อมูลได้ทั้งเป็น PCDATA หรือ child element X, Y ก็ได้

นอกจากนี้ XML ยังใช้สัญลักษณ์ต่างๆ เข้ามาเป็นตัวช่วยในการประกาศอิลิเมนต์ โดยสามารถสรุปเป็นตารางได้ดังนี้

สัญลักษณ์	ตัวอย่าง	ความหมาย
(	(X,Y)	อิลิเมนต์จะต้องมีลำดับของเนื้อหา (content) เป็น X และ Y
,	(X,Y,Z)	ตามลำดับ
	(X Y Z)	อิลิเมนต์ต้องมีเนื้อหาเป็น X, Y และ Z
?	X?	อิลิเมนต์ต้องมีเนื้อหาเป็น X หรือ Y หรือ Z อย่างใดอย่างหนึ่ง
*	X*	อิลิเมนต์อาจมีเนื้อหาเป็น X และหากมีเนื้อหาเป็น X ก็จะต้องปรากฏได้เพียงครั้งเดียวเท่านั้น

+	X+	อิลิเมนต์อาจมีเนื้อหาเป็น X และหากมีเนื้อหาเป็น X ก็ สามารถปรากฏได้มากกว่า 1 ครั้ง
ไม่มีสัญลักษณ์	X	อิลิเมนต์จะต้องมีเนื้อหาเป็น X อย่างน้อย 1 ครั้งขึ้นไป อิลิเมนต์จะต้องมีเนื้อหาเป็น X

ตารางที่ 3-1 แสดงสัญลักษณ์ต่างๆใช้ในการประกาศอิลิเมนต์

เราจะมาดูตัวอย่างของการใช้สัญลักษณ์ต่างๆดังนี้

```
<!ELEMENT Email(To+,From,CC*,BCC*,Subject?,Body?)>
```

ซึ่งมีความหมายว่า อิลิเมนต์ที่ชื่อ Email ประกอบด้วยรายการของอิลิเมนต์อื่นๆ ได้แก่ To, From, CC, BCC, Subject และ Body โดยที่

- To จะต้องมียูเอชทีเอ็มแอลอย่างน้อย 1 คำ
- From จะต้องมียูเอชทีเอ็มแอลได้เพียง 1 คำ
- BCC และ CC อาจจะมีหรือไม่มียูเอชทีเอ็มแอลก็ได้ ถ้ามีก็สามารถมีได้แค่ 1 คำ
- Subject อาจจะมีหรือไม่มียูเอชทีเอ็มแอลก็ได้ แต่ถ้ามีก็สามารถมีได้แค่ครั้งเดียว
- Body อาจจะมีหรือไม่มียูเอชทีเอ็มแอลก็ได้ แต่ถ้ามีก็สามารถมีได้แค่ครั้งเดียว

3.5 DTD Comments

การเขียนคอมเมนต์ในส่วนของ DTD จะมีลักษณะเหมือนกับการเขียนคอมเมนต์ใน XML นั่นคือข้อความที่จะเป็นคอมเมนต์จะต้องอยู่ระหว่างเครื่องหมาย <!-- และ -->

3.6 การกำหนด DTD ไว้ในเอกสาร XML

การประกาศใช้งาน DTD มี 2 แบบ คือ ประกาศไว้ในเอกสาร XML และ ประกาศไว้เป็นไฟล์นามสกุล DTD ซึ่งแยกกันอยู่กับเอกสาร XML จากตัวอย่างที่ผ่านมาเราจะประกาศ DTD ไว้ในเอกสาร XML การที่เราจะกำหนด DTD ไว้ในเอกสาร XML นั้นเราจะใช้ DOCTYPE ดังนี้

```
<!DOCTYPE rootelement SYSTEM URL>
```

3.6.1 Public Document Type Definitions

เมื่อเรามี DTD ที่ไว้ใช้กับการใช้ทั่วไป เราจะใช้คีย์เวิร์ด PUBLIC มาแทน SYSTEM ที่อยู่ใน <!DOCTYPE> document type declaration ซึ่งจะใช้รูปแบบดังนี้

```
<!DOCTYPE rootelement PUBLIC identifier URL>
```

การใช้ศัพท์เวิร์ด PUBLIC เราจะต้องสร้าง formal public identifier (FPI) และกำหนดกฎให้กับ FPI ดังนี้

- ในส่วนแรกเป็นส่วนของการติดต่อของ DTD สำหรับ DTD ที่เราเป็นคนกำหนดเองในส่วนนี้ควรเป็นค่า “-“ ถ้า nonstandard body ถูกต้องตามแบบแผนของ DTD ใช้ “+” และสำหรับรูปแบบที่เป็นมาตรฐาน ในส่วนนี้จะอ้างถึงค่ามาตรฐานของมัน (อย่างเช่น ISO/IEC13449:2000)
- ในส่วนที่สองต้องใส่ชื่อของกลุ่มหรือนุคคลที่คอยดูแล หรือรับผิดชอบ DTD ในที่นี้ควรใช้ชื่อที่ไม่ซ้ำกัน (unique) และอ้างถึงกลุ่มได้ง่าย เช่น W3C ควรใช้ w3c
- ในส่วนที่สามต้องแสดงรูปแบบของเอกสารที่อธิบาย ควรที่จะตามด้วย unique identifier ของ ชนิด เช่น version 1.0 ซึ่งในส่วนนี้ควรประกอบด้วยตัวเลขของ version ที่คุณจะ update
- ในส่วนที่สี่ จะเป็นส่วนที่กำหนดภาษาที่ DTD ใช้ เช่นภาษาอังกฤษจะใช้ EN
- แต่ละส่วนของ FPI ต้องแยกด้วย double slash (//)

```
<!DOCTYPE Email PUBLIC “-//kmitl//Student XML version 1.0//EN”
“email.dtd”>
```

3.7 DTD Entities

เอนทิตีใน XML มีอยู่ 2 ชนิด คือ general entities และ parameter entities โดย general entities จะอยู่ใน content ของเอกสาร XML ซึ่งจะขึ้นต้นด้วยเครื่องหมาย & และลงท้ายด้วยเครื่องหมาย ; ส่วน parameter entities จะใช้ในเอกสาร DTD ซึ่งจะเริ่มต้นด้วยเครื่องหมาย % และลงท้ายด้วยเครื่องหมาย ;

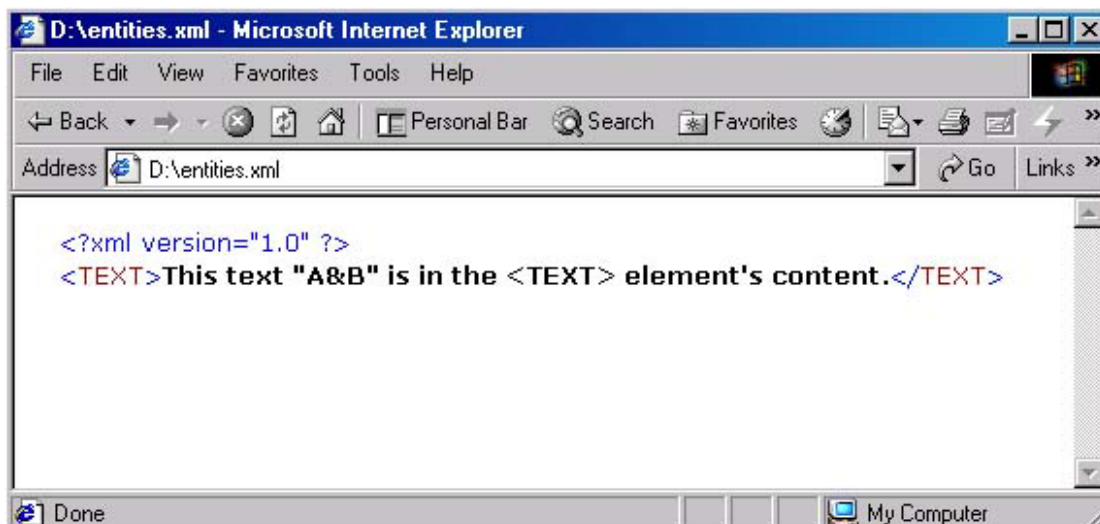
เอนทิตี สามารถเป็นได้ทั้ง parsed และ unparsed เอนทิตี ซึ่งใน content ของ parsed เอนทิตี จะเป็นข้อความที่เป็น well-formed XML ส่วน unparsed เอนทิตีจะเก็บข้อมูลที่ไม่ต้องการ parsed เช่น ข้อมูลทั่วไปหรือ ข้อมูลที่เป็นไบนารี

3.7.1 General Entities

ปกติแล้ว XML จะ define เอนทิตีพื้นฐานไว้แล้ว 5 ตัวคือ <, >, &, " และ ' เอนทิตีเหล่านี้จะใช้แทนตัวอักษร <, >, &, “ และ ‘ ตามลำดับ ซึ่งเราไม่ต้อง defined มันลงไปใน DTD อีก จากตัวอย่างจะเป็นการใช้เอนทิตีที่มีการ define ไว้แล้ว 5 ตัวดังนี้

```
<?xml version="1.0" ?>
<TEXT>
    This text &quot;A&amp;B&quot; is in the &lt;TEXT&gt; element&apos;s content.
</TEXT>
```

เมื่อทำการ parsed แต่ละเอนทิตีจะถูกแทนด้วยตัวอักษรที่ถูกกำหนดค่าไว้ในแต่ละ entities โดย XML processor โดยในรูปที่ 3-1 จะแสดงเอกสารนี้ที่เปิดใน Internet Explorer สังเกตว่าทุก entities จะถูกแทนด้วยค่าของแต่ละ entities ดังนี้



รูปที่ 3-1 แสดงผลลัพธ์การใช้ general เอนทิตีใน Internet Explorer

เราสามารถ define เอนทิตีของเราเองได้โดยการ declare ไว้ใน DTD ซึ่งเราจะใช้อิเลเมนต์ `<!ENTITY>` ในการ declare เอนทิตี (คล้ายๆกับการใช้ `<!ELEMENT>`) โดยจะใช้รูปแบบดังนี้

```
<!ENTITY NAME DEFINITION>
```

โดย NAME จะเป็นชื่อของเอนทิตี และ DEFINITION เป็นค่าของเอนทิตี ตัวอย่างต่อไปนี้จะเป็นการแสดงการใช้เอนทิตีที่ง่ายที่สุด ในที่นี้เราจะตั้งชื่อของเอนทิตีว่า KMITL และค่าของเอนทิตีเป็นชื่อเต็มของ KMITL ดังนี้

```
<?xml version="1.0" ?>
<!DOCTYPE TEXT [
  <!ENTITY KMITL "King Mongkut's Institute of Technology Ladkrabang"
]>
<TEXT>&KMITL;</TEXT>
```

จากตัวอย่างข้างต้นเราจะได้ผลลัพธ์ที่แสดงผ่าน Internet Explorer ดังนี้ สังเกตว่าที่ `&KMITL` ใน source code เมื่อทำการ parsed แล้วจะถูกแทนที่ด้วยค่า (definition) ที่อยู่ในเอนทิตี KMITL



รูปที่ 3-2 แสดงผลลัพธ์การใช้ general เอนทิตีที่สร้างขึ้นเองใน Internet Explorer

3.7.2 Parameter Entities

parameter entity สามารถใช้ได้ในการเอกสาร DTD เท่านั้น การสร้าง parameter entity นั้นคล้ายกับการสร้าง general entity โดยจะมี % อยู่ภายในอิลิเมนต์ `<!ENTITY>` ดังนี้

```
<!ENTITY % NAME DEFINITION>
```

และนี่เป็นตัวอย่างการใช้ internal parameter entity ซึ่งเราจะ declare เอนทิตีชื่อว่า BR และมี definition คือ `<!ELEMENT BR EMPTY>` และเวลาเรียกใช้งานเราจะขึ้นต้นด้วย % ตามด้วยชื่อของเอนทิตี และลงท้ายด้วย ; ดังนี้

```
<?xml version="1.0" ?>
<!DOCTYPE TEXT [

<!ENTITY %BR "<!ELEMENT BR EMPTY">
<!ENTITY KMITL "King Mongkut's Institute of Technology Ladkrabang"
%BR;
]>
<TEXT>&KMITL;</TEXT>
```

3.8 Attributes

ทุกอิลิเมนต์ใน XML สามารถที่จะใส่ attribute ต่างๆได้ และเราสามารถที่จะกำหนดได้ว่าแต่ละอิลิเมนต์ในเอกสาร XML ควรที่จะมี attributes ชื่ออะไร และมีลักษณะเป็นแบบไหน มีแค่ดีฟอลต์ (default) เป็นอะไรก็ได้ โดยใช้อิลิเมนต์ `<!ATTLIST>` ในการกำหนดค่าลิสต์ของ attribute ดังนี้

```
<!ATTRIBUTE ELEMENT_NAME
    ATTRIBUTE_NAME TYPE DEFAULT_VALUE
    ATTRIBUTE_NAME TYPE DEFAULT_VALUE
```

โดยที่ ELEMENT_NAME คือชื่อของอิลิเมนต์ที่เราจะ declare attributes, ATTRIBUTE_NAME คือชื่อของ attribute ที่จะ declare, TYPE คือ ชนิดของ attribute และ DEFAULT_VALUE เป็นการกำหนดค่าดีฟอลต์ให้กับ attribute ซึ่งตารางข้างล่างนี้จะเป็นตารางที่สรุปค่าของ TYPE ทั้งหมดที่มี

TYPE	Description
CDATA	เป็นตัวอักษรธรรมดาที่ไม่ประกอบด้วย ตัวอักษรที่เป็น markup
ENTITIES	ชื่อเอนทิตีหลายตัว(ซึ่งต้อง declare ใน DTD)แยกกันด้วยช่องว่าง(whitespace)
ENTITY	ชื่อเอนทิตี (ซึ่งต้อง declare ใน DTD)
Enumerated	แทน list of values (ต้องใช้ค่าใดค่าหนึ่งจาก list)
ID	เป็นคุณสมบัติของ XML ที่ชื่อจะต้องไม่ซ้ำกัน นั่นคือแต่ละ attribute ที่มีค่า ID จะมีชื่อที่ไม่ซ้ำกัน
IDREF	จะถือค่าของ ID attribute ของบางอิลิเมนต์ โดยปกติอิลิเมนต์ที่เป็นอิลิเมนต์ปัจจุบัน(current element) จะเกี่ยวข้อง
IDREFS	แสดง ID ของอิลิเมนต์หลายค่าซึ่งแยกโดยช่องว่าง(whitespace)
NMTOKEN	แสดงชื่อ XML ที่เกี่ยวข้อง
NMTOKENS	แสดงชื่อของ XML ที่เกี่ยวข้องกันหลายค่าในลิสต์ ซึ่งแยกกันโดยช่องว่าง(whitespace)
NOTATION	แสดงชื่อของ notation ซึ่งต้อง declare ใน DTD

ตารางที่ 3-2 สรุปค่าต่างๆของ TYPE ใน Attributes

ในส่วนของตารางด้านล่างนี้จะเป็นตารางที่สรุปค่า DEFAULT_VALUE ที่เป็นไปได้ทั้งหมดดังนี้

Method	Description
VALUE	แสดงข้อมูลพื้นฐานซึ่งถูกปิดด้วยเครื่องหมายคำพูด
#IMPLIED	แสดงว่า attribute นี้ไม่มีค่าดีฟอลต์ และ จะมีหรือไม่มีค่านี้ก็ได้
#REQUIRED	แสดงว่า attribute นี้ไม่มีค่าดีฟอลต์ แต่จะต้องใส่ค่าให้ attribute นี้เสมอ
#FIXED VALUE	ในที่นี้ VALUE คือค่าของ attribute และ attribute นี้จะต้องใช้ค่านี้เสมอ

ตารางที่ 3-3 สรุปค่าต่างๆของ DEFAULT_VALUE ที่อยู่ใน Attributes